

Alineacion de tablas y separacion de CSS

Ventas-Frontend ·
Compras-Frontend-Local

Documento de trabajo — que se cambio, por que, y como verificarlo.

1. Que se cambio

Hoy se corrigio la alineacion de cuatro tablas. El sintoma era siempre el mismo: los titulos de las columnas no caian encima de sus datos. En algunos casos el titulo se iba a la derecha y el numero al centro; en otros, el titulo se quedaba a la izquierda mientras los botones de la columna quedaban al centro.

La causa no era cada tabla: era una pelea entre reglas CSS de distinta especificidad. La seccion 2 lo explica completo; el resto del documento es el cambio archivo por archivo.

ARCHIVO	QUE SE TOCO
css/cajas.css	Tabla del modal de cajas: las 4 columnas centradas
reportes.css	.sesiones-table (reporte de ventas por caja): todo centrado
salehistory.css	.money y th.money centradas
compras.css	.money y th.money centradas

Ademas se documenta en este PDF el trabajo previo de separacion del CSS del modal de cajas (seccion 6) y se deja constancia de dos temas que ya estaban resueltos y que conviene no volver a abrir: la paginacion del carrito (seccion 7) y los campos obligatorios (seccion 8).

2. La causa de fondo: especificidad CSS

styles.css centra, desde hace tiempo, TODAS las tablas que viven dentro de una tarjeta .table-card. Eso incluye los titulos y los datos:

src/styles.css (lineas 315 y 333)

```
.table-card th {
  text-align: center;
}
.table-card td {
  text-align: center;
}
```

Cada componente despues redefine la alineacion de sus columnas de monto. El problema es que esas redefiniciones NO tienen el mismo peso que el global, y por eso unas ganaban y otras no. Una regla mas especifica gana siempre; si empatan, gana la que se escribio despues.

SELECTOR	PESO	GANA AL GLOBAL?	RESULTADO REAL
.table-card td	(0,1,1)	—	centro
.table-card th	(0,1,1)	—	centro
.money	(0,1,0)	NO: pierde	centro (el global)
th.money	(0,1,1)	empate: gana por orden	DERECHA
.sesiones-table .money	(0,2,0)	SI	DERECHA
.sesiones-table thead th	(0,1,2)	SI	IZQUIERDA

Lo que producía el desajuste

En el historial de ventas y en compras, .money se aplicaba a los <td> y a los <th> por igual. Sobre el papel deberían verse iguales, pero no:

- <td class="money"> lo resolvía .table-card td, de peso (0,1,1), que le gana a .money, de peso (0,1,0). El dato quedaba CENTRADO.
- <th class="money"> lo resolvía th.money, de peso (0,1,1): empate exacto con .table-card th, así que ganaba por orden de carga. El título quedaba a la DERECHA.

O sea: el mismo nombre de clase producía dos alineaciones distintas segun si estaba en un th o en un td. Ese es exactamente el "título corrido" que se veía en pantalla.

El caso de reportes era el contrario

En .sesiones-table las reglas del componente si ganaban (0,2,0) y (0,1,2), así que título y dato SI coincidían: los dos a la derecha. La tabla no estaba descuadrada, pero su columna de montos quedaba corrida respecto a las demás, que si seguían el centro del global.

La regla que se aplica de aquí en adelante

Cuando una columna lleva una clase que define su alineación, esa clase se declara en el th Y en el td, con el mismo valor, y no se apoya en el global. Se declara de mas, a proposito: si el global cambia mañana, la tabla no se descuadra sola. Y el padding se deja simétrico, porque con el texto centrado un padding solo de un lado lo descentra.

3. Los cambios, archivo por archivo

3.1 src/app/features/charge/css/cajas.css

La tabla del modal "Ver cajas". El <th> "Ultimo" no traía alineación propia, así que heredaba el left de .boxes-table th; y "Acciones" estaba a la derecha mientras los botones se centraron con justify-content.

```
.boxes-table th {
-   text-align: left;
+   text-align: center;
    font-size: 0.72rem;
}
```

```
.boxes-table th.fecha-col,
.boxes-table td.fecha-col {
+   /* CENTRADA: antes el <th> "Ultimo" heredaba `left`. */
+   text-align: center;
    white-space: nowrap;
}
```

```
.boxes-table th.acciones-col,
.boxes-table td.acciones-col {
-   text-align: right;
+   text-align: center;
}

.boxes-table th.caja-col,
.boxes-table td.caja-col {
-   text-align: left;
+   text-align: center;
}
```

```
.boxes-actions {
    display: flex;
    gap: 6px;
-   justify-content: flex-end;
+   justify-content: center;
    align-items: center;
}
```

Nota: .boxes-actions es un <div> dentro del <td>, no el <td> mismo. La sección 6 explica por qué esa diferencia es obligatoria y no un detalle de estilo.

3.2 src/app/features/reports/reportes.css

Reporte de ventas por caja: se elige la caja y la tabla lista sus turnos (sesiones-table, 9 columnas). Aquí el problema era el contrario al del historial: las reglas del componente ganaban y dejaban montos a la derecha y los demás títulos a la izquierda.

```

.sesiones-table thead th {
  /* ... */
-  text-align: left;
+  text-align: center;
  padding: 9px 10px;
}

.sesiones-table thead th.money {
-  text-align: right;
+  text-align: center;
}

```

```

.sesiones-table .money {
-  text-align: right;
+  text-align: center;
  font-variant-numeric: tabular-nums;
  white-space: nowrap;
}

```

La columna Tickets (.num) ya estaba centrada en ambos lados y no se tocó. El font-variant-numeric: tabular-nums se conserva: aunque el texto se centre, los dígitos siguen ocupando el mismo ancho, que es lo que evita que la columna "bailen" al cambiar de 9 a 10.

3.3 src/app/features/salehistory.css

Historial de ventas. Aquí es donde se veía más claro el descuadre: el título a la derecha y el monto al centro.

```

.money {
-  text-align: right;
+  text-align: center;
  font-variant-numeric: tabular-nums;
  padding-right: 18px !important;
+  padding-left: 18px !important;
}

th.money {
+  text-align: center;
+  padding-right: 18px !important;
+  padding-left: 18px !important;
}

```

th.money antes solo traía padding-right. Se le agregó text-align: center explícito porque al empatar con .table-card th lo que decidía era el orden de carga de los styleUrls, y eso es frágil. .col-id (número de ticket) y .col-fecha (04/10/2026 14:30, con white-space: nowrap) ya funcionaban y no se tocaron.

3.4 src/app/features/shopping/compras.css

Compras tiene tres tablas: el historial de compras, el detalle expandible y la tabla de márgenes. El mismo cambio que en el historial de ventas.

```

.money {
-   text-align: right;
+   text-align: center;
    font-variant-numeric: tabular-nums;
    padding-right: 18px !important;
+   padding-left: 18px !important;
}

th.money {
-   text-align: right;
+   text-align: center;
    padding-right: 18px !important;
+   padding-left: 18px !important;
}

```

Este unico cambio tambien corrige la subtabla de detalle de cada compra, porque sus cuatro columnas de monto (Costo unit., Precio venta, Margen, Subtotal) usan la misma clase .money. La columna % (th.pct, td.pct) ya estaba centrada en ambos lados y no se toco.

4. Por que el padding se vuelve simetrico

Al centrar el texto, un padding que existe solo a la derecha deja de ser invisible: empuja el contenido hacia el lado contrario al espacio que ocupa.

```

.money {
    text-align: center;
    padding-right: 18px; /* imbalancedo: descentra a la izquierda */
}

.money {
    text-align: center;
    padding-right: 18px;
    padding-left: 18px; /* simetrico: centro real de la celda */
}

```

Los !important se conservan porque ya estaban ahi y estan peleando con el padding del global (.table-card td). Quitarlos seria un segundo cambio en la misma linea; conviene hacerlo aparte y probarlo.

5. Que NO se toco, y por que

Quedaron cuatro reglas de alineacion que no son de las cuatro tablas pedidas. Se listan para que no se confundan con un olvido:

REGLA	ARCHIVO	MOTIVO
.sesion-detalle .subtable th	reportes.css:687	Subtabla del detalle del turno
.subtable th	compras.css:422	Subtabla del detalle de la compra
.linea-cabecera span:nth-child(5)	compras.css:579	Encabezado del carrito del modal
.modal-total	compras.css:832	El total grande del modal: a la derecha esta bien

La decision de fondo: se centran las COLUMNAS de datos de las tablas que se pidieron. Un encabezado que no es una columna de tabla (como el total de un modal) se queda donde se lee mejor, que en ese caso es a la derecha.

6. La separacion del CSS del modal de cajas

Trabajo previo, de la misma tanda. La pregunta era si conviene partir el CSS de cobro en varios archivos. Si: asi quedo.

6.1 El <td> no puede ser el contenedor flex

Este es el punto que mas se olvida. La version anterior ponía los botones directamente en el <td> y le aplicaba display: flex para poder ponerlos en fila.

```
<td class="acciones-col">
  <div class="boxes-actions">
    <button class="box-btn-edit">Editar</button>
    <button class="box-btn-del">Eliminar</button>
  </div>
</td>
```

Un <td> con display: flex deja de comportarse como celda de tabla: deja de participar del algoritmo de reparto de columnas. A partir de ahí, table-layout: fixed deja de mandar y el ancho de la columna deja de respetarse: la tabla se ensancha y los botones se salen del modal. El <div> interior si puede ser flex, y si puede hacer flex-wrap.

6.2 Los anchos van en porcentaje, no en pixeles

```
.boxes-table {
  table-layout: fixed;
}

.boxes-table th.caja-col,      .boxes-table td.caja-col    { width: 30%; }
.boxes-table th.turnos-col,   .boxes-table td.turnos-col { width: 12%; }
.boxes-table th.fecha-col,    .boxes-table td.fecha-col  { width: 24%; }
.boxes-table th.acciones-col, .boxes-table td.acciones-col { width: 34%; }
```

Con table-layout: fixed, el navegador IGNORA min-width y max-width de las celdas. Por eso los anchos se declaran en porcentaje y en un solo lugar. La version anterior declaraba la columna de acciones dos veces (una en pixeles al principio y otra "de correccion" al final del archivo); la segunda no hacia nada y solo confundia.

6.3 overflow: hidden no arregla nada

La regla que mas dano hizo: overflow: hidden en la celda de acciones. Cuando los botones no entaban, los recortaba, y el sintoma (un boton sin boton) desaparecia mientras el problema seguia. Recortar el desbordamiento no es arreglar: es tapar el sintoma. Con los anchos en porcentaje y flex-wrap: wrap en el <div>, los botones bajan de linea y se siguen viendo.

6.4 Una colision de clases que si era un bug

.btn-del existia en dos sitios con reglas incompatibles: el boton de quitar un renglon del carrito del POS y el boton "Eliminar" de una caja. Dependiendo de la hoja que ganara, uno aparecia rojo y el otro naranja.

CLASE	DONDE	COMO SE RESOLVIO
.btn-del	carrito del POS	Se dejo como estaba
.box-btn-del	modal de cajas	Prefijo propio (box-btn-)
.box-btn-edit	modal de cajas	Prefijo propio (box-btn-)
.box-btn-baja	modal de cajas	Prefijo propio (box-btn-)
.box-btn-alta	modal de cajas	Prefijo propio (box-btn-)

7. La paginacion del carrito ya existia

Este punto se pidio y quedo RESUELTO de antemano.
Se deja constancia de como esta, para no volver a implementarlo.

QUE	DONDE	VALOR
Tamano de pagina	sale-facade.ts:222	cartPageSize = 5
Pagina actual	sale-facade.ts:209	cartPage = 0
Recorte de filas	cobro.html:132	pipe PaginatePipe
Pie de paginacion	cobro.html:174-179	app-pagination
Ir a la pagina del producto	sale-facade.ts:238	irAPaginaDelProducto()
Ajustar al quitar	sale-facade.ts:264	ajustarPaginaAlQuitar()

cobro.html (los parentesis alrededor del async NO son opcionales)

```
<tr *ngFor="let item of  
  (sale.cobroItems$ | async) | paginate: sale.cartPage : sale.cartPageSize">
```

Sin los parentesis, Angular lo lee como a | (async | paginate), que no existe, y el build falla.
Por eso el PaginatePipe acepta T[] | null | undefined: el pipe async devuelve null antes del primer valor.

Los tres detalles que hacen que se comporte bien:

- `irAPaginaDelProducto()` al agregar, incluido el escaner: escanear un producto que cae en la pagina 2 lo agregaba invisible, y el carrito "no cambiaba" a ojos del cajero.
- `ajustarPaginaAlQuitar()` al quitar o bajar: borrar el ultimo renglon de la ultima pagina dejaba la vista en una pagina que ya no existe, y el carrito se veia vacio.
- `cartPage = 0` tras `cobro.clear()`: si no, despues de cobrar un carrito que estaba en la pagina 3, la venta siguiente aparece "en la pagina 3" de un carrito de 1 producto.

8. Los campos obligatorios

Tambien pedido y ya implementado, en los dos lados.

No se toco codigo en esta sesion.

8.1 Frontend: `core/utills/validadores.ts`

Validadores compartidos, sin dependencias de Angular, que los cinco formularios ya importan: `categorias.ts:15`, `roles.ts:16`, `usuarios.ts:16`, `compras.ts:29` (el CRUD de proveedores vive aqui, no en una carpeta propia) y `productos.ts:31`.

<code>validarSku(valor)</code>	<code>validarBarcode(valor)</code>	<code>validarRfc(valor)</code>
<code>validarEntero(valor)</code>	<code>validarPrecio(valor)</code>	<code>validarEmail(valor)</code>
<code>validarTexto(v, campo, max)</code>	<code>validarTelefono(valor)</code>	<code>validarPassword(valor)</code>
<code>soloDigitos(event)</code>	<code>aMayusculas(event)</code>	<code>aMinusculas(event)</code>

Cada funcion devuelve `ValidationResult` con el mensaje, y cada formulario muestra el error EN EL CAMPO, no como una ventana emergente al apretar Guardar. `productos.ts` ya reporta con `MatSnackBar` (`productos.ts:168`).

8.2 Backend: commit `a672cef` "Validacion de campos"

QUE	DONDE
<code>requerido(valor, campo) / requerida(p, campo)</code>	<code>InputValidator.java</code>
Contraseña obligatoria (8-16)	<code>InputValidator.java</code>
Nombre, correo y contrasena obligatorios	<code>UserImpl.java</code>
SKU y codigo de barras obligatorios	<code>ProductEntity.java</code> + servicios
NOT NULL en la base	<code>V7__campos_obligatorios.sql</code>
294 tests en verde	<code>InputValidatorTest</code> + <code>UserImplCamposObligatoriosTest</code>

Dos detalles del backend que conviene conocer:

- La comparacion es contra `isBlank()`, no contra `!= null`. Un campo con tres espacios ya es vacio; un `!= null` lo daria por bueno.
- `requerido()` y `requerida()` existen por el genero: "El contrasena es obligatorio" estaba mal, y una prueba de API lo cazó. Se lee "La contrasena es obligatoria".

El frontend avisa antes de enviar y el backend sigue siendo la garantía real. No es duplicación: el frontend señala DONDE está el error; el backend es el que impide que un campo obligatorio llegue a guardarse como null.

8.3 alert() o snackbar: cual es cual

	ALERT()	MATSNACKBAR
Bloquea la pantalla	SI	NO
Estilo	Nativo del navegador	De la app (Material)
Desaparece solo	NO: hay que cerrarla	SI (duration)
Varios errores juntos	Uno por vez, encadenados	Uno por vez, sin encadenar
Botones extra	OK / Cancelar	La acción que se le pase
Se ve en un celular	Molesto	Normal

Regla que se sigue en el proyecto: alert() y confirm() para lo que es decisión y detiene el flujo (dar de baja una caja, eliminar, cerrar un turno). MatSnackBar para informar que algo pasó o está mal, sin robar la pantalla. Los formularios de login, recuperación de contraseña usan snackbar; los CRUD usan alert, y esa mezcla se queda pendiente de homogeneizar.

9. Verificación

COMANDO	RESULTADO
npx ng build --configuration development	OK (11.5 s, 1.68 MB)
npx ng test --watch=false	No encuentra specs (ver abajo)

Los tests no corren por el entorno, no por el código: la ruta del repositorio tiene parentesis ("Sistema de ventas (comida)") y eso rompe el patrón que busca los archivos de prueba. Es un problema conocido de esta máquina.

Revisión manual sugerida: abrir el modal de cajas con varias filas, el reporte de ventas eligiendo una caja con turnos, el historial de ventas y la pestaña de márgenes en compras. En las cuatro, cada título debe caer sobre su columna y los montos deben quedar centrados y con los signos \$ alineados.